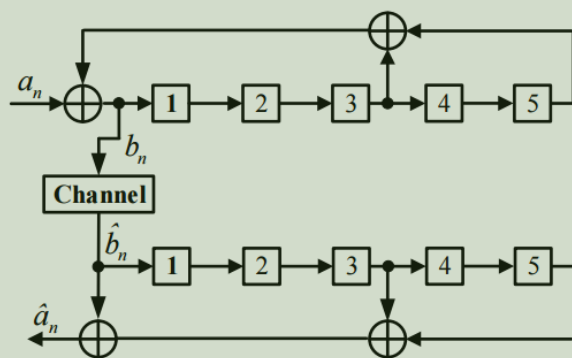


為什麼要 Scrambling ?

為了「時脈恢復」 (Clock Recovery) 這是最重要的原因。在非同步傳輸中，接收端必須透過訊號中的「位元轉換」(由 0 變 1 或由 1 變 0) 來對齊發送端的時脈。所以如果原始資料出現很長一段連續的 000000... 或 111111...，訊號線上的電位會一直保持不變。導致接收端找不到變化的時間點，時脈發生偏移，進而解錯資料。擾碼器會將長串的 0 或 1 攪亂，增加 0 與 1 之間的切換頻率，確保接收端能隨時校正時脈。

Design Description:

1. The pseudorandom polynomial $f(x) = x^5 + x^3 + 1$
2. The block diagram of the self-synchronization scrambler/descrambler is shown as below.



3. The initial values of $\{b_{n-1}, b_{n-2}, b_{n-3}, b_{n-4}, b_{n-5}\}$ and $\{\hat{b}_{n-1}, \hat{b}_{n-2}, \hat{b}_{n-3}, \hat{b}_{n-4}, \hat{b}_{n-5}\}$ are equivalent to $\{1, 1, 1, 1, 1\}$ and $\{0, 0, 1, 0, 0\}$, respectively.
4. The format of the input sequences is described as $a_n = [5, 0, A, 5, 1, A, 5, 2, A, 5, 3, A, \dots]$, where all values in a_n are hexadecimal.
5. The channel is assumed to be ideal, namely, $h_n = 1$.

我先把輸入 [5, 0, A, 5, 1, A, 5, 2, A, 5, 3, A] 轉成二進制

[0101, 0000, 1010, 0101, 0001, 1010, 0101, 0010, 1010, 0101, 0011, 1010]

初始狀態：Scrambler：1, 1, 1, 1, 1，Descrambler：0, 0, 1, 0, 0

解擾碼器的暫存器長度為 5，在接收到 5 個 bit 後，解擾碼器的暫存器狀態會與擾碼器一樣。從第 6 位 (n=6) 開始，輸出會正確還原輸入。

程式碼

```
clear; clc;

a_bin = [0,1,0,1, 0,0,0,0, 1,0,1,0, 0,1,0,1, 0,0,0,1, 1,0,1,0,
0,1,0,1, 0,0,1,0, 1,0,1,0, 0,1,0,1, 0,0,1,1, 1,0,1,0];
N = length(a_bin);

reg_s = [1, 1, 1, 1, 1];
reg_d = [0, 0, 1, 0, 0];

s_data = zeros(N, 8);
d_data = zeros(N, 7);

for n = 1:N
    % --- Scrambler ---
    bn = xor(a_bin(n), xor(reg_s(3), reg_s(5)));
    s_data(n, :) = [n, a_bin(n), reg_s, bn];
    reg_s = [bn, reg_s(1:4)];

    bn_hat = bn;

    % --- Descrambler ---
    an_hat = xor(bn, xor(reg_d(3), reg_d(5)));
    d_data(n, :) = [n, reg_d, an_hat];
    reg_d = [bn_hat, reg_d(1:4)];
end

fid1 = fopen('Scrambler.txt', 'w');
fprintf(fid1, '%s Scrambler (f(x) = x^5 + x^3 + 1)\n');
fprintf(fid1, ' n | an | (1) (2) (3) (4) (5) | bn\n');
fprintf(fid1, '---|---|-----|---\n');
for n = 1:N
    fprintf(fid1, '%2d | %d | %d %d %d %d %d | %d\n',
s_data(n, :));
end
fclose(fid1);
```

```

fid2 = fopen('Descrambler.txt', 'w');
fprintf(fid2, '%% Descrambler Process (Initial: 00100)\n');
fprintf(fid2, ' n | (1) (2) (3) (4) (5) | an_hat\n');
fprintf(fid2, '---|-----|-----\n');
for n = 1:N
    fprintf(fid2, '%2d | %d %d %d %d %d | %d\n',
d_data(n, :));
end
fclose(fid2);

fprintf('look Scrambler.txt and Descrambler.txt\n');
bin2hex_str = @(b) sprintf('%X', bin2dec(reshape(char(b +
'0'), 4, []).'));
fprintf('原始輸入 : %s\n', bin2hex_str(a_bin));
fprintf('擾碼輸出 : %s\n', bin2hex_str(s_data(:, 8)'));
fprintf('解擾輸出 : %s\n', bin2hex_str(d_data(:, 7)'));

```

輸出結果

```
look Scrambler.txt and Descrambler.txt
原始輸入 (Hex): 50A51A52A53A
擾碼輸出 (Hex): 42EF947E8110
解擾輸出 (Hex): E8A51A52A53A
fx >>
```

% Scrambler ($f(x) = x^5 + x^3 + 1$)								% Descrambler Process (Initial: 00100)							
n	an	(1)	(2)	(3)	(4)	(5)	bn	n	(1)	(2)	(3)	(4)	(5)	an_hat	
1	0	1	1	1	1	1	0	1	0	0	1	0	0	1	
2	1	0	1	1	1	1	1	2	0	0	0	1	0	1	
3	0	1	0	1	1	1	0	3	1	0	0	0	1	1	
4	1	0	1	0	1	1	0	4	0	1	0	0	0	0	
5	0	0	0	1	0	1	0	5	0	0	1	0	0	1	
6	0	0	0	0	1	0	0	6	0	0	0	1	0	0	
7	0	0	0	0	0	1	1	7	0	0	0	0	1	0	
8	0	1	0	0	0	0	0	8	1	0	0	0	0	0	
9	1	0	1	0	0	0	1	9	0	1	0	0	0	1	
10	0	1	0	1	0	0	1	10	1	0	1	0	0	0	
11	1	1	1	0	1	0	1	11	1	1	0	1	0	1	
12	0	1	1	1	0	1	0	12	1	1	1	0	1	0	
13	0	0	1	1	1	0	1	13	0	1	1	1	0	0	
14	1	1	0	1	1	1	1	14	1	0	1	1	1	1	
15	0	1	1	0	1	1	1	15	1	1	0	1	1	0	
16	1	1	1	1	0	1	1	16	1	1	1	0	1	1	
17	0	1	1	1	1	0	1	17	1	1	1	1	0	0	
18	0	1	1	1	1	1	0	18	1	1	1	1	1	0	
19	0	0	1	1	1	1	0	19	0	1	1	1	1	0	
20	1	0	0	1	1	1	1	20	0	0	1	1	1	1	

可以看到從第六個 clock cycle 開始，input 與 output 資料一樣了!

達成同步